



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

POSTGRESQL PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Databases

TOTAL: 10 QUESTIONS

Q1 What type of database is PostgreSQL and what are its core strengths? Architecture

Q2 How does PostgreSQL guarantee consistency and handle transactions? Architecture

Q3 What index types does PostgreSQL support and when do you use each? Architecture

Q4 How do you design a schema or data model in PostgreSQL? Architecture

Q5 How does PostgreSQL scale horizontally? SSO / IdP

Q6 How do you monitor and tune slow queries in PostgreSQL? Ability

Q7 What backup and restore strategies does PostgreSQL support? Lifecycle

Q8 How do you handle schema migrations in PostgreSQL? Compliance

Q9 What security features does PostgreSQL provide? Testing

Q10 What are common anti-patterns when using PostgreSQL in production? Performance



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 PostgreSQL is a relational, document, key-value, graph, or time-series store. **Mitigation**

PostgreSQL is a relational, document, key-value, graph, or time-series store. Its strengths such as ACID guarantees, horizontal scale, or flexible schema guide the workloads it is best suited for.

A6 PostgreSQL provides a slow query log, explain/analyze, or profiling tools. **Observability**

PostgreSQL provides a slow query log, explain/analyze plan, or profiling tools to surface inefficient queries. Common fixes include adding indexes, rewriting queries, or increasing cache size.

A2 PostgreSQL provides either full ACID transactions or eventual consistency. **Primitives**

PostgreSQL provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Transaction support varies from none (simple K/V stores) to serializable isolation (relational DBs).

A7 PostgreSQL supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. **Automation**

PostgreSQL supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Point-in-time recovery requires WAL/oplog archiving on top of backups.

A3 PostgreSQL offers B-tree, hash, full-text, spatial, or composite indexes. **Hardening**

PostgreSQL offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Choose based on query patterns: B-tree for range queries, hash for equality lookups, full-text for search.

A8 Schema migrations in PostgreSQL are managed with versioned migration files. **Governance**

Schema migrations in PostgreSQL are managed with versioned migration files applied in order by a migration tool. Migrations should be idempotent and tested in staging before production to avoid downtime.

A4 Schema design in PostgreSQL involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). **Relationships**

Schema design in PostgreSQL involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Understanding query needs upfront prevents costly migrations later.

A9 PostgreSQL supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. **Encryption**

PostgreSQL supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Always restrict user privileges to the minimum needed and disable anonymous access in production.

A5 PostgreSQL scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. **Federation**

PostgreSQL scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Some PostgreSQL implementations handle this automatically; others require manual configuration.

A10 Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage. **Optimization**

Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage.